
NanoGPT from Scratch

Pooja Laxmi Sankarakameswaran
sankarakameswaran.p@northeastern.edu
College of Engineering,
Northeastern University

Jiaying Zheng
zheng.jiay@northeastern.edu
Khoury College of Computer Science,
Northeastern University

Poornima Jaykumar Dharamdasani
jaykumardharamdasani.p@northeastern.edu
Khoury College of Computer Science,
Northeastern University

Abstract

This project presents the development and evaluation of NanoGPT, a lightweight and efficient adaptation of the GPT architecture designed to generate coherent and contextually relevant text with minimal computational overhead. NanoGPT was trained and tested across three diverse datasets: the Tiny Shakespeare Dataset for generating Shakespeare-like text, the Spotify Million Song Dataset for generating song lyrics, and the Story Cloze Dataset for generating 5-sentence stories based on prompts. The model’s performance was evaluated using perplexity as the primary metric, with results demonstrating a range of capabilities across different text generation tasks. The project also provides insights into the hyperparameters, data split, and tokenization techniques employed, offering a comprehensive understanding of NanoGPT’s performance across multiple tasks.

1 Introduction

Transformer, with the self-attention mechanism, has fundamentally changed the landscape of text generation by allowing models to understand and generate long-range dependencies in text more effectively than previous architectures [1]. Recent advancements have further refined Transformer-based models, enabling more controlled and context-aware text generation, addressing challenges such as coherence and stylistic consistency [2]. The rapid advancement of Generative Pre-trained Transformers (GPT) has revolutionized natural language processing (NLP) by significantly enhancing the ability to generate coherent and contextually relevant text across a wide range of applications. This has led to the development of powerful models like ChatGPT, which have gained immense popularity for their ability to produce human-like text. These models have become a cornerstone of modern NLP due to their versatility and effectiveness across various domains. However, the widespread adoption of such models is often hindered by their substantial computational requirements, which limit accessibility and scalability. This challenge has prompted the exploration of lightweight alternatives that can achieve similar levels of performance while being more applicable.

1.1 Problem Statement

Given the rising demand for ChatGPT-like text generation models, this project aims to build a lightweight version of ChatGPT, named NanoGPT from scratch. The objective is to implement a model that retains the core capabilities of ChatGPT—such as generating coherent and contextually relevant text—while being optimized for deployment on diverse datasets with minimal computational overhead.

NanoGPT will be designed to perform various text generation tasks, including the creation of Shakespearean dialogues, song lyrics, and generating stories. By focusing on a more streamlined architecture, the project will demonstrate that it is possible to implement a smaller, yet powerful, model from scratch that can deliver high-quality outputs across different use cases. This approach seeks to bridge the gap between the need for advanced text generation capabilities and the practical limitations of deploying large models, making such technology more accessible for a broader range of applications.

1.2 Proposed Approach

The proposed method involves implementing a NanoGPT model from scratch, which is a lightweight version of the ChatGPT architecture utilizing the core components of the Transformer model. The specifics of the model architecture, as well as the experiments performed and the evaluation of these models, are thoroughly discussed in the following sections.

2 Dataset

Three different datasets have been used in this work, to pre-train the model on three different text generation tasks.

- **Tiny Shakespeare Dataset [6]:**
 - **Content:** Comprises 40,000 lines from a variety of Shakespeare’s works, including plays and poems. This extensive collection captures a broad spectrum of Shakespeare’s literary style.
 - **Use:** Employed primarily in natural language processing tasks focused on text generation and the study of Shakespearean language patterns.
- **Spotify Million Song Dataset [4]:**
 - **Content:** This dataset includes a CSV file containing information on 643 artists, 44,824 song names, 57,650 links to songs, and lyrics for 57,494 songs.
 - **Use:** Utilized in a variety of music-related data science tasks, from predictive analytics to text generation, especially valuable for projects involving lyrical content.
- **Story Cloze Dataset [5]:**
 - **Content:** Contains approximately 98,161 five-sentence stories designed for the evaluation of story understanding and narrative completion abilities in models.
 - **Use:** Critical for developing and testing models on narrative prediction and common-sense reasoning within the context of story generation.

These datasets provide a broad spectrum of linguistic and stylistic challenges, enabling the model to learn diverse patterns of text generation, from poetic and dramatic texts to musical lyrics and structured stories.

3 Method

3.1 Model Architecture

Most competitive neural sequence transduction models utilize an encoder-decoder architecture [1]. However, for text generation tasks such as language modeling, only the decoder block is typically employed. This is because the network needs to generate future text based on previously generated text, acting autonomously without the input processing typically handled by an encoder. This project also uses a ‘decoder-only’ architecture (Figure 1), where the decoder predicts the next word in a sequence given the words that preceded it.

3.2 Decoder Stack

The decoder is composed of a stack of N identical layers. Each layer has two sub-layers. The first is a masked multi-head self-attention mechanism, and the second is a simple, position-wise

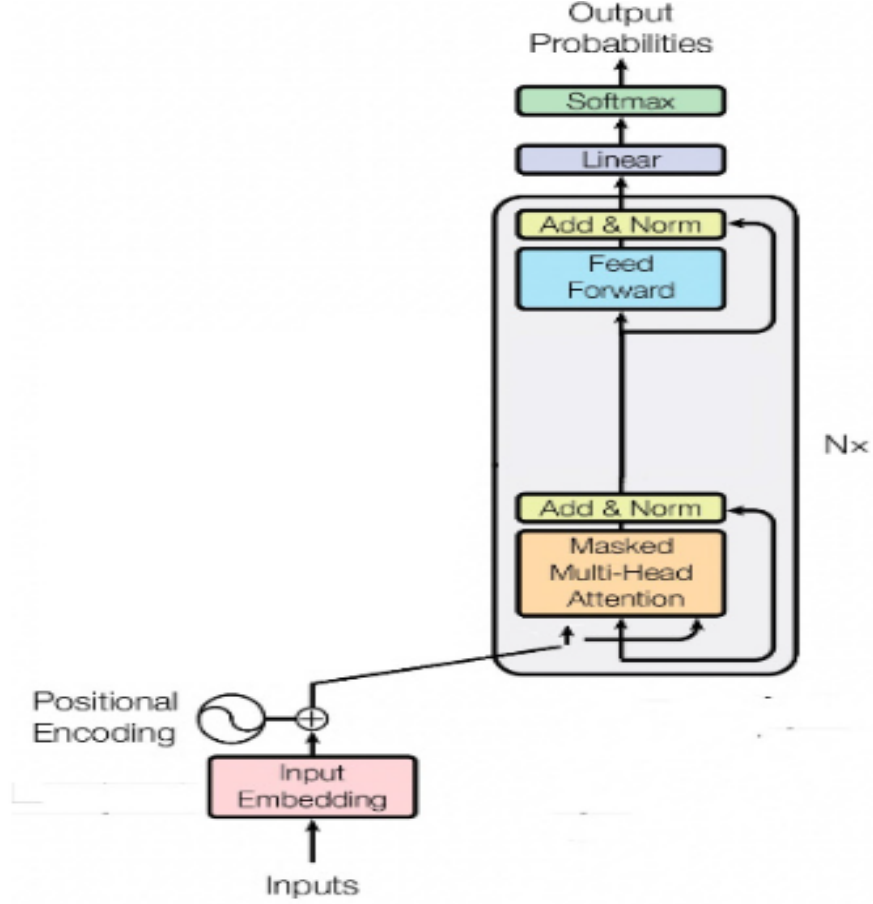


Figure 1: The Decoder-only Transformer Model Architecture.

fully connected feed-forward network. It has residual connections around each of the sub-layers, followed by layer normalization. The self-attention sub-layer is to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i .

3.3 Attention

An attention function can be characterized as mapping a query alongside a collection of key-value pairs to produce an output, with the query, keys, values, and output all being vector representations. The output is determined by a weighted sum of these vectors.

3.3.1 Scaled Dot-Product Attention

The input of the Scaled Dot-Product Attention (Figure 2) layer consists of queries and keys of dimension d_k , and values of dimension d_v . The dot product of the query is computed with all the keys, and divided by $\sqrt{d_k}$, and then a softmax function is applied to obtain the weights on the values.

In practice, the attention function is computed simultaneously on a set of queries, packed together into a matrix Q . The keys and values are also packed together into matrices K and V . The matrix of outputs is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

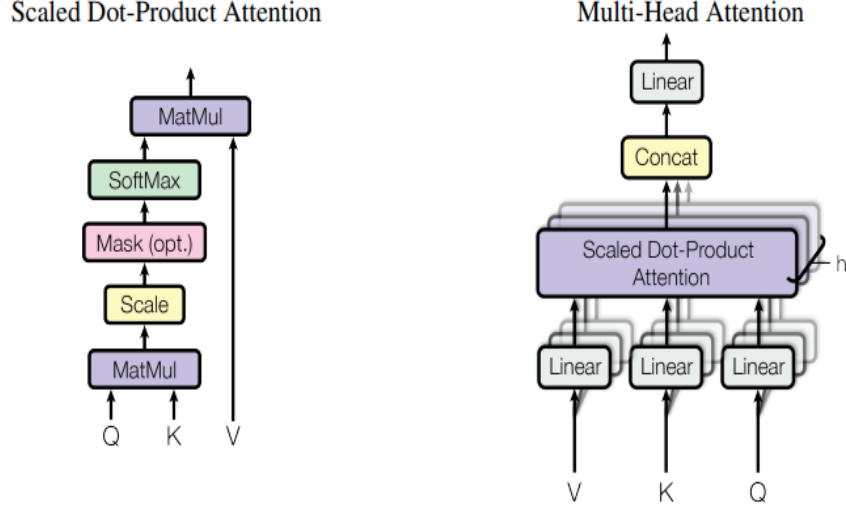


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

3.3.2 Multi-Head Attention

Instead of performing a single attention function with d_{model} -dimensional keys, values and queries, it is beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k , d_k , and d_v dimensions, respectively. On each of these projected versions of queries, keys and values the attention function is performed in parallel, yielding d_v -dimensional output values. These are concatenated and once again projected, resulting in the final values, as depicted in Figure 2. Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.

In this work the number of heads have been tuned depending on the application. The self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. To prevent leftward information flow in the decoder all values in the input of the softmax which correspond to illegal connections have been masked out in the scaled dot-product attention by setting to $-\infty$.

3.4 Position-wise Feed-Forward Networks

In addition to attention sub-layers, each of the layers in the decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

While the linear transformations are the same across different positions, they use different parameters from layer to layer.

3.5 Embeddings and Softmax

The learned embeddings are used to convert the input tokens and output tokens to vectors of dimension d_{model} . The linear transformation and softmax function are used to convert the decoder output to predicted next-token probabilities.

3.6 Positional Encoding

Since the model has no convolution layers, in order for the model to make use of the order of the sequence, "positional encodings" are added to the input embeddings at the bottoms of the decoder stack. The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed.

3.7 Applications of the proposed architecture

The decoder-only model has been applied to three different text generation tasks as listed below:

1. Shakespeare Dialog Generation Using the Tiny Shakespeare Dataset:

- **Objective:** The goal here is to generate text that mimics the unique style and complex language structures of William Shakespeare. This involves understanding and reproducing Shakespearean vocabulary, syntax, and dramatic dialogue.
- **Challenges:** The model must capture the rich, archaic vocabulary and intricate rhetorical structures typical of Shakespeare’s work, which are significantly different from contemporary English.

2. Song Lyric Generation Using the Spotify Million Song Dataset:

- **Objective:** To generate song lyrics that are lyrical, rhyming, and rhythmically consistent, reflecting the styles of various artists represented in the dataset.
- **Challenges:** Lyrics need to not only be meaningful and contextually appropriate but also conform to the constraints of rhyme and rhythm that define good songwriting. Additionally, the model needs to adapt to the diverse styles and emotional tones of different music genres.

3. Story Generation Using the Story Cloze Dataset:

- **Objective:** To create complete narratives that are logically coherent and engaging, where the model must predict a plausible story ending from a given story context.
- **Challenges:** This task tests the model’s ability to understand narrative structure, causal relationships, and character development over a longer text span than the other two applications.

In all these cases, the decoder-only architecture leverages its capability to generate text from learned context, making it particularly suited for tasks where the generation of coherent, context-aware text based on previous tokens is crucial.

4 Experimentation

This section provides a comprehensive overview of the experimental methodologies employed to optimize the NanoGPT model across three distinct text generation tasks: Tiny Shakespeare, Lyric Generator and Story Generator. We detail the hyperparameter tuning strategies, data splitting procedures, tokenization techniques and training processes utilized to enhance model performance and adaptability for each specific dataset.

4.1 Hyperparameter Tuning

In this study, we performed extensive hyperparameter tuning to optimize the NanoGPT model for three different text generation tasks: Shakespearean text generation, song lyric generation, and 5-sentence story generation. Each task required a tailored approach to achieve the best performance, considering the unique characteristics of the respective datasets. All the hyperparameters detailed below were determined through a series of experiments with various values, as illustrated in Table 1. The selection of the optimal hyperparameters was guided by the perplexity score (which is explained in Section 5).

For the Tiny Shakespeare Dataset, the model was configured with an embedding dimension of 384 and 6 attention heads. The batch size for training was set to 64, and the vocabulary size was capped at 65 tokens. The final model contains 10.65 million parameters.

In the Spotify Million Song Dataset experiment, the model’s embedding dimension was reduced to 128 to accommodate the shorter and more varied nature of song lyrics. The number of attention heads was set to 4, with a batch size of 64. The vocabulary size was increased to 77, and the total parameter count was 0.83 million.

For the Story Cloze Dataset, the model’s hyperparameters were set to an embedding dimension of 512, with 4 attention heads, a batch size of 512, and a vocabulary size of 10,000. The model contained 22.8 million parameters.

4.2 Data Splitting

To ensure robust evaluation of the NanoGPT model, each dataset was divided into training and validation sets. All of the datasets were divided with 80% of the data used for training and 20% for validation. These splits allowed us to monitor the model’s performance on unseen data and adjust the hyperparameters accordingly.

4.3 Tokenization

The choice of tokenizer played a critical role in preparing the datasets for training. For the Tiny Shakespeare Dataset and the Spotify Million Song Dataset, we opted for a character-level tokenizer. This approach was particularly effective in capturing the nuanced stylistic elements inherent in Shakespeare’s text and song lyrics. The character-level tokenizer allowed the model to learn the finer details of text structure, which is crucial for generating text that closely mimics the original style.

In contrast, the Story Cloze Dataset required a more sophisticated tokenization strategy due to the complexity of the stories. We employed Byte-Pair Encoding (BPE), which balances the trade-off between vocabulary size and token granularity. BPE was chosen for its ability to efficiently handle the variety of words and phrases in the dataset, enabling the model to generate more coherent and contextually appropriate stories.

4.4 Training Process

For the Shakespearean text generation use case, the training process involved minimizing the cross-entropy loss function using the Adam optimizer, with a learning rate set to $1e-3$. The model was trained on one L4 GPU, which expedited the process and allowed for handling larger batch sizes. The evaluation was conducted using perplexity, a metric that quantifies the model’s uncertainty in predicting the next token.

In the song lyric generation use case, the training also utilized the Adam optimizer with a learning rate set to $1e-3$. Similar to the Shakespearean text generator, this model was trained on 2 T4 GPUs to ensure efficient training with a batch size optimized for the dataset. Perplexity was used to assess model performance.

For the 5-sentence story generation based on the Story Cloze Dataset, the training approach was consistent with the other use cases, employing the Adam optimizer and a learning rate set to $1e-4$. The model was trained on 1 P100 GPU, which facilitated quicker iterations and larger batch sizes. Perplexity was the primary evaluation metric.

5 Results

The NanoGPT model’s performance was evaluated across three text generation tasks. The primary evaluation metric used was perplexity [3], which measures the model’s ability to predict the next word in a sequence. Lower perplexity scores indicate better performance, and the generated text samples provide additional insights into the model’s capabilities. The results are summarized in the Table 2.

5.1 Tiny Shakespeare

The model achieved a perplexity score of 3.84 on the Tiny Shakespeare Dataset, indicating strong performance in generating text that mimics the style of Shakespeare. The generated text captured the

Table 1: Different hyperparameters experimented along with the perplexity scores. The final hyperparameters chosen have been bolded. n_head is the number of attention heads, n_embed is the embedding dimension.

Use Case	batch_size	n_embed	n_head	Perplexity Score
Tiny Shakespeare	64	384	8	13.11
	32	768	8	18.37
	64	384	6	3.84
Lyric Generation	16	64	4	4.3
	32	64	4	3.86
	64	128	4	3.08
Story Generation	128	128	6	71.6
	64	64	2	124.1
	512	512	4	42.6

Table 2: Evaluation results

Use Case	Perplexity Score
Tiny Shakespeare	3.84
Lyric Generation	3.08
Story Generation	42.6

archaic language, rhythmic patterns, and thematic elements characteristic of Shakespearean dialogue. However, while the text demonstrated an understanding of Shakespeare’s linguistic style, some phrases were syntactically irregular and semantically unclear, suggesting areas for improvement in generating more coherent passages. An example snippet of the dialogues generated by the model is shown in Figure 3.

5.2 Lyric Generator

In the song lyric generation task, the model achieved the lowest perplexity score of 3.08, reflecting its strong capability in producing coherent and stylistically appropriate lyrics. The generated lyrics maintained a clear structure and adhered to the informal, colloquial tone common in contemporary music. Despite these strengths, some lines exhibited syntactical inconsistencies and unusual word choices, which occasionally led to nonsensical phrases. Overall, the model demonstrated a solid understanding of lyrical composition, with minor areas that could benefit from further refinement. A sample verse generated by the model is shown below in Figure 4.

5.3 Story Generator

The story generation task presented the greatest challenge for the model, as evidenced by a perplexity score of 42.6. The generated text showed significant issues with coherence and logical flow, resulting in narratives that lacked depth and contextual understanding. The model struggled to maintain a consistent storyline, leading to disjointed and unclear passages. This suggests that the current model configuration is less effective for tasks requiring complex narrative generation, and further tuning or alternative approaches may be necessary to improve performance in this area. A 5 sentence story generated by the prompt ‘Little girl’ is shown below in Figure 5.

6 Discussion

The results from the three implementations—Shakespeare Dialogue Generator, Lyric Generator, and Story Generator—demonstrate that the NanoGPT model is capable of generating text that adheres to the expected format of each specific task after being trained on corresponding datasets. However, the performance across these tasks highlights several limitations and areas for improvement.

KING RICHARD II:
 Say they stir and the best of Saint George,
 Take the happiness of them wed last,
 But when I defy them his mother,
 And for their vain to the begging their injuries
 And leave their flowers in raises like death.
 HENRY BOLINGBROKE:
 Marry, let me leave the foot of the house.
 HENRY PERCY:
 What, what evil sorrow this? what news?
 HENRY BOLINGBROKE:
 Madam, repent to the Duke of Norfolk,
 Unto the Earl of Lancaster; and as the hand and bear
 What statute can can all the happy from her c

 Men pardon, mark thee and gapest of a children:
 Some hours that any thousand head and us,
 In this scorn something casely to England,
 My worst shall be be thou to dead.
 BUCKINGHAM:
 O, then, is the guilty fellow's neighbour
 Of channibals, like a needle garland,
 Hath written country's the least self-monarchy well not
 To dear the city, and till be the noise of thee,
 Though to this often now the sun of all the city;
 And so the silent were beloved to him there,
 And all this own to report; you cannot

Figure 3: Tiny Shakespeare text

Women better again
 Take a mean-easlesper and bean,
 The boy turns you have about thrills of catches in on

 I didn't need a new boy to,
 I didn't need a little you go again
 I don't need a touch the game boys
 I didn't need a one million front,
 The true lockeds up and with this baby where's mine, baby where's mine,
 I know about this order night and victorial my days,
 But when this moon starin' wbo, it didn't already

 Oh, baby, didn't what, painter bitch is just up
 But I wish you cry the gave main games
 Got a post question, postch the door in mash, and end to the night the door inside
 Girl I'll ride for your curses
 I'll hold for your traveli's tender the famish
 Scraping a main child baby
 I gotta lying leave for a country to an empty ry
 You cant a say here singing leflau leave for me there

Figure 4: Lyric Generator text


```
'Little girl'  
Little Girl Recini Joce Jill entered a baking competition.  
First, she was waiting for her start.  
It was Halloween and Handleg careless.  
She saw a scary bed of clowns.  
When the sun was there when her mom found out her.
```

Figure 5: Story Generator

6.1 Lyric Generator Issue

While the Lyric Generator successfully produced song lyrics in the correct format, an issue arose where some of the generated words were not valid English words. This could be attributed to the model's reliance on a character-level tokenizer, which, while effective at capturing fine-grained text patterns, may also generate out-of-vocabulary or nonsensical words, especially when trained on a relatively small dataset. To address this, one potential improvement could involve expanding the training dataset with more diverse and extensive lyric data, ensuring that the model encounters a broader range of vocabulary during training.

6.2 Story Generator Issue

The Story Generator, while producing text in the correct narrative format, frequently generated stories that lacked logical coherence. This issue is likely due to the inherent complexity of storytelling, which requires the model to maintain context and logical consistency over longer text spans. The use of Byte Pair Encoding (BPE) tokenizer was intended to help with handling longer sequences, but the relatively small size of the training dataset might have limited the model's ability to learn coherent long-range dependencies. To mitigate this, future work should consider training on larger, more diverse story datasets that provide richer examples of narrative structure. Additionally, experimenting with more advanced training techniques, such as curriculum learning where the model is gradually introduced to increasingly complex stories, could improve logical consistency.

6.3 Impact of Hyper-parameters

Adjusting hyperparameters had a noticeable impact on model performance; however, it also led to a higher risk of overfitting. This overfitting is likely due to the limited size of the datasets, which constrains the model's ability to generalize.

In summary, while the NanoGPT model shows promise in generating text across different formats, the current limitations underscore the importance of dataset size and quality, as well as the careful tuning of model parameters. Future work should focus on addressing these challenges by leveraging larger datasets and exploring alternative evaluation methods to better capture the nuances of text generation tasks.

References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS’17). Curran Associates Inc., Red Hook, NY, USA, 6000–6010.
- [2] Hanqing Zhang, Haolin Song, Shaoyu Li, Ming Zhou, and Dawei Song. 2023. A Survey of Controllable Text Generation Using Transformer-based Pre-trained Language Models. *ACM Comput. Surv.* 56, 3, Article 64 (March 2024), 37 pages. <https://doi.org/10.1145/3617680>
- [3] Arora, Kushal and Anand Rangarajan. “Contrastive Entropy: A new evaluation metric for unnormalized language models.” *arXiv: Computation and Language* (2016): n. pag.
- [4] <https://www.kaggle.com/datasets/notshrirang/spotify-million-song-dataset>
- [5] https://huggingface.co/datasets/LSDSem/story_cloze
- [6] <https://www.kaggle.com/datasets/thedevastator/the-bards-best-a-character-modeling-dataset>

7 Appendix

The following subsections outline the structure and execution details of the Python notebooks used for all three use cases. These sections serve as a comprehensive guide to understanding and executing the code, providing essential insights into the implementation and execution process.

7.1 Structure of the Code

The notebooks are organized into two distinct sections: **Training Process** and **Demo**. The **Training Process** section encompasses all components related to the transformer architecture, including the training and evaluation phases. In contrast, the **Demo** section provides the necessary code for generating and retrieving text outputs.

7.2 External Dependencies Libraries

- `pytorch`
- `numpy`
- `transformers`
- `datasets`

7.3 Class Descriptions

1. `Head(nn.Module)`:

- **Description:** The `Head` class implements a single self-attention head in a transformer model, transforming input sequences using `key`, `query`, and `value` linear layers to compute attention scores. These scores are masked with a lower triangular matrix to prevent attending to future tokens, ensuring the model maintains an autoregressive property. After applying softmax to normalize the scores and dropout to prevent overfitting, the class performs a weighted aggregation of the `value` vectors, producing an output tensor that captures contextual relationships within the input sequence. The output retains the same dimensions as the input, enriched with contextual information from the attention mechanism.

2. `MultiHeadAttention(nn.Module)`:

- **Description:** The `MultiHeadAttention` class models multiple parallel self-attention heads, which are combined to enhance the model's ability to focus on different parts of the input sequence simultaneously. The class is initialized with a specified number of attention heads, each implemented as an instance of the `Head` class and stored in a `ModuleList`. During the forward pass, the outputs from each head are concatenated along the embedding dimension and then projected through a linear layer, followed by dropout to prevent overfitting. The final output is a contextually enriched representation of the input, benefiting from the diverse perspectives provided by multiple attention heads.

3. `FeedFoward(nn.Module)`:

- **Description:** The `FeedForward` class implements a simple yet effective feedforward neural network module, typically used within transformer models. It consists of a sequence of layers, starting with a linear transformation that expands the input embedding dimension (`n_embd`) to four times its size, followed by a ReLU activation function to introduce non-linearity. This is followed by another linear layer that reduces the dimensionality back to the original embedding size. Finally, a dropout layer is applied to the output to prevent overfitting. The entire sequence of operations is encapsulated in a `Sequential` container, which is applied to the input tensor during the forward pass, enabling the model to learn complex representations of the data.

4. `Block(nn.Module)`:

- **Description:** The `Block` class represents a fundamental building block of a transformer model, encapsulating both self-attention and feedforward neural network components. It first applies layer normalization to the input, then passes it through a multi-head self-attention mechanism (`MultiHeadAttention`), and adds the result back to the original

input (a process known as a residual connection). This output is again normalized before being passed through a feedforward network (`FeedForward`), and the result is added back to the input, forming a second residual connection. This combination of self-attention for communication and feedforward layers for computation allows the transformer block to capture complex dependencies in the input data while maintaining stability during training through normalization and residual connections.

5. `BigramLanguageModel(nn.Module)`:

- **Description:** The `BigramLanguageModel` class is a transformer-based language model designed to predict the next token in a sequence based on the current context. It begins by embedding input tokens and their positions using `nn.Embedding` layers, which convert them into dense vectors. These embeddings are summed and passed through a series of transformer blocks (`Block`), each consisting of self-attention and feedforward sublayers, to capture contextual dependencies. After processing through these blocks, the output is normalized using layer normalization and then projected to the vocabulary size using a linear layer to generate logits for the next token prediction. The model also includes a loss calculation using cross-entropy for training and a `generate` function that iteratively predicts and appends new tokens to the input sequence, allowing the generation of text sequences of arbitrary length. This architecture enables the model to learn and generate text sequences effectively by leveraging the power of self-attention and residual connections.

7.4 Function Descriptions for the training code

1. `get_batch(split)`:

- **Description:** This function generates a batch of input-output pairs for training or validation. It utilizes memory-mapped files to load data efficiently and prepares batches by selecting random indices for the input data. Depending on the execution device (CPU or GPU), it handles data pinning for asynchronous data transfer to the GPU.

2. `estimate_loss()`:

- **Description:** A function to estimate the average loss over a specified number of iterations (`eval_iters`) for both training and validation splits. It operates in evaluation mode to prevent updates to model parameters and uses no gradient calculation to save memory and computational resources. The function returns a dictionary containing the mean loss for each data split.

3. `get_lr(it)`:

- **Description:** This function computes the learning rate based on the iteration number. It supports three phases: linear warmup, constant maximum learning rate, and cosine decay to a minimum learning rate. The actual learning rate applied depends on whether the current iteration is in the warmup phase, the decay phase, or beyond the total number of decay iterations.

7.5 Function Descriptions for the testing code

The testing code includes the process of generating samples from a trained GPT model, including model loading, sample generation, and output configuration. The main components of the code are outlined below:

1. **Model Initialization:** Loads a model either from a saved checkpoint (`'resume'`) or by initializing from a pre-trained GPT-2 variant. It adjusts the model's configuration based on the initialization type and compiles the model if specified for faster execution.
2. **Device and Dtype Configuration:** Sets up the device (CPU or CUDA) and data type for tensors, enabling TensorFlow 32-bit types if supported, and configures automatic mixed precision context if necessary.
3. **Encoding and Decoding Setup:** Configures text encoding and decoding based on the model's training data, using either stored metadata or default GPT-2 encoding mechanisms.

4. **Sample Generation:** Encodes an initial text prompt, generates multiple text samples based on the specified generation parameters (like temperature and top-k tokens), and prints each sample to the console.

7.6 Input

The input varies across different use cases. For both the Tiny Shakespeare and Lyric Generator tasks, the number of tokens to be generated is specified as input. Conversely, for the Story Generator task, the input consists of a prompt provided to the generate_story function.

7.7 Output

The output is formatted as dialogues for the Tiny Shakespeare task, verses for the Lyric Generator, and complete sentences for the Story Generator.

7.8 Unit test cases & Demo

Tiny Shakespeare Use case: The unit tests for the Tiny Shakespeare dataset are included in the train.py and model.py scripts, respectively. The demonstration code is located in the test.py script. Detailed instructions for running the codes can be found in the readme file of the GitHub repository. The link to the code for this use case is :<https://github.com/Agzjy/NLP-Project-Team105/tree/master/NanoGPT%20Models/Tiny-Shakespeare>.

Lyrics Generator Use case: The unit tests for the Lyrics Generator model are included in the unit_test.ipynb scripts, respectively. The demonstration code is located in the demo.ipynb script. Detailed instructions for running the codes can be found in the readme file of the GitHub repository. The link to the code for this use case is :[https://github.com/Agzjy/NLP-Project-Team105/tree/master/NanoGPT%20Models/Lyrics Generator](https://github.com/Agzjy/NLP-Project-Team105/tree/master/NanoGPT%20Models/Lyrics%20Generator).

Story Generator Use case: The unit tests for the Story Generator model are included in the unit_test.py and the demo code is included in the demo.py file. Both of these files and all the code for this usecase can be found in the Story Generator folder in the Google Drive Link : <https://drive.google.com/drive/folders/1sSMrExLluIJpIHNABM7rID8fprvvZqUf>.

7.9 Instructions to execute the Code

Tiny Shakespeare dataset: The main scripts are:

- 1) Demo script: test.py
- 2) Data preparation script: prepare.py
- 3) Training script: train.py

The detailed instructions for running the codes can be found in the readme file of the GitHub repository. The link to the code for this use case is :<https://github.com/Agzjy/NLP-Project-Team105/tree/master/NanoGPT%20Models/Tiny-Shakespeare>.

Lyrics Generator (Spotify Million Song dataset): The main scripts are:

- 1) Demo script: demo.ipynb
- 2) Data & Model script: lyric_generator_model.ipynb
- 3) Unit test script: unit_test.ipynb

The detailed instructions for running the codes can be found in the readme file of the GitHub repository. The link to the code for this use case is :[https://github.com/Agzjy/NLP-Project-Team105/tree/master/NanoGPT%20Models/Lyrics Generator](https://github.com/Agzjy/NLP-Project-Team105/tree/master/NanoGPT%20Models/Lyrics%20Generator).

Story Generator (Story Cloze Dataset): The main scripts are:

- 1) Data & Model script : spm.model, story_generator.ipynb
- 2) Demo Script : demo.py
- 3) Unit Test : unit_test.py

The detailed instructions for running the codes can be found in the readme file of the GitHub repository. The link to the code for this use case is <https://github.com/Agzjy/NLP-Project-Team105/tree/master/NanoGPT>